



晶創25使用者服務介紹

V1.0

台灣智慧雲端服務股份有限公司
Taiwan AI Cloud Corporation

講師: 劉政杰

大綱

- 課前準備
- HPC功能介紹
- Slurm 簡介
- Module 概觀
- Miniconda 介紹
- Apptainer / Singularity容器介紹
- 跨節點運算跟單節點運算
- Q & A 與補充

課前準備

註冊iService會員 ☒

帳號/主機帳號

加入有效專案 ☒

專案在效期內、額度 > 0 、專案啟用晶創25服務

ssh操作工具 ☒

MobaXterm(建議), VScode, putty ...

大綱

- 課前準備
- **HPC功能介紹**
- Slurm 簡介
- Module 概觀
- Miniconda 介紹
- Apptainer / Singularity 容器介紹
- 跨節點運算跟單節點運算
- Q & A 與補充

晶創25 (Nano5)

晶創25(英文名稱：Nano 5)為新一代加速器叢集式超級電腦，適用於產業界、政府機構、學術單位以及研究機構，為各類用戶提供高效能的運算支援，助力AI全面推動。

- 服務簡介

晶創25主機為台灣發展AI及高效能運算的關鍵基礎設施。晶創主機也承襲「晶創計畫」的願景，助力半導體產業發展AI應用，也為半導體產業其他各種應用及研發，提供強大且高速的計算環境。該系統採用NVIDIA Hopper計算加速器，總算力將達16 PetaFLOPS，可大幅提升人工智慧模型訓練與科學模擬的運算效率。

晶創25 主機介紹

,21 台NVIDIA HGX H100計算節點

- 2顆Intel Xeon Platinum 8480+ 2.0GHz處理器 (或是 8480CL)
- **8顆NVIDIA H100 GPU**
- 8 Port NVIDIA InfiniBand 400Gb/s網路埠
- 2TB Memory

• 16 台NVIDIA HGX H200計算節點

- 2顆Intel Xeon Platinum 8480+ 2.0GHz處理器
- **8顆NVIDIA H200 GPU**
- 8 Port NVIDIA InfiniBand 400Gb/s網路埠
- 2TB Memory

共可提供296顆NVIDIA Hopper計算加速器，實際整體最佳效能約為19.01PFlops，可大幅提升人工智慧模型訓練與科學模擬的運算效率

晶創25 主機介紹

- 計算資源與設置
 - 整體效能最高達19.01Petaflops
 - 10 PB儲存容量
 - 機房位置：國家高速網路與計算中心台中分部
 - 時間校正：系統時間已校時至國網中心NTP Server
 - 作業系統：皆採用 RHEL 8.10

晶創25 主機連線介紹

登入節點與傳輸節點

類別	網域名稱	描述
登入節點	nano5.nchc.org.tw:22	ssh登入後可使用命令列執行下列工作: 1. 提交、管理job 2. 可存取高速檔案系統上的檔案 3. 編譯HPC相關應用程式 4. 運行除錯程式以改善程式碼
資料傳輸節點	nano5.nchc.org.tw:2222	可使用資料傳輸工具執行下列工作: 1. 資料傳入/傳出高速檔案系統(HFS) 2. 支援 sftp

晶創25 主機連線介紹

- 連線登入
 - 連線方式 : ssh
 - 登入節點 : nano5.nchc.org.tw:22
 - 帳號 : <主機帳號>
 - 密碼 : <主機密碼>
 - OTP : <OTP>

※ OTP : One Time Password

修改主機帳號基本資料

主機帳號

主機密碼

OTP 載具: 有效

主機帳號

變更主機密碼

刪除使用者載具

最近一次變更主機密碼日期: 2024-04-03 20:10:06

OTP載具資訊係由第三方(narlabs.org.tw)提供。

注意: 已發送APP 載具註冊通知信件

使用 IDExpert 手機 APP(APP OTP 或 Push)進行登入節點雙因子(2FA) 驗證作業前，請至 信箱收取載具註冊通知信件，並依內容指示完成 APP 載具註冊作業。

使用 On-Demand進行登入節點雙因子(2FA) 驗證作業，可自下方區塊調整收取一次性OTP驗證碼的通知方式。

▶OTP通知: 僅電子郵件

▶電子郵件:

編輯

晶創25 主機連線介紹

● 設定HFS空間額度

- iService (https://iservice.nchc.org.tw/nchc_service/index.php)

- 點選 會員中心



設定高速檔案系統HFS



高速檔案系統HFS(Nano5/Nano4共用)



NCHC 高速檔案系統(HFS)目錄空間管理系統

iService
計算資源服務網

會員中心

服務介紹

操作說明

常見

會員資訊

計畫管理

申購管理

設定高速檔案系統HFS

計算主機負載資訊

帳號基本資料

高速檔案系統
HFS(TWCC/T3 共用)

高速檔案系統HFS(F1)

高速檔案系統
HFS(Nano5/Nano4共用)

注意：已發送A
使用 IDEpe

晶創25 HFS

HFS User Portal >> 晶創25-x86 個人管理

空間資訊

用量紀錄

異動紀錄

會員帳號:

會員名稱:

HFS 空間資訊

目錄	最大可用空間	已用空間	剩餘空間
Home	100GB	1GB (1%)	99GB (99%)
Work	100GB	0GB (0%)	100GB (100%)

平台

晶創25

計算節點架構

x86

登入

晶創25 功能介紹

HFS空間額度查詢

```
hfsquota
```

```
[twsutwx533@cbi-lgn01 ~]$ hfsquota
PATH                USED                HARD LIMIT    USAGE %    STATUS
/home/twsutwx533    499,712 B    107,374,182,400 B    0    ACTIVE
/work/twsutwx533    2,240,512 B    107,374,182,400 B    0    ACTIVE
```

Lab 1：如何開始使用HPC

1. 登入iService會員中心確認OTP載具
2. 使用 ssh 工具連線至登入節點(nano5.nchc.org.tw:22)
3. 使用傳輸軟體連線至資料傳輸節點(nano5.nchc.org.tw:2222)
4. 確認HFS空間額度查詢
5. 使用 wallet 指令查看有效錢包

大綱

- 課前準備
- HPC功能介紹
- **Slurm 簡介**
- Module 概觀
- Miniconda 介紹
- Apptainer / Singularity容器介紹
- 跨節點運算跟單節點運算
- Q & A 與補充

晶創25 功能介紹

Hardware

晶創25 共有37台GPU計算節點 (H100X21 & H200X16)，提供使用者以高性能的資源來完成運算工作，因此需要一個具有合理規範的叢集調度系統，來負責任務的調配。

Software

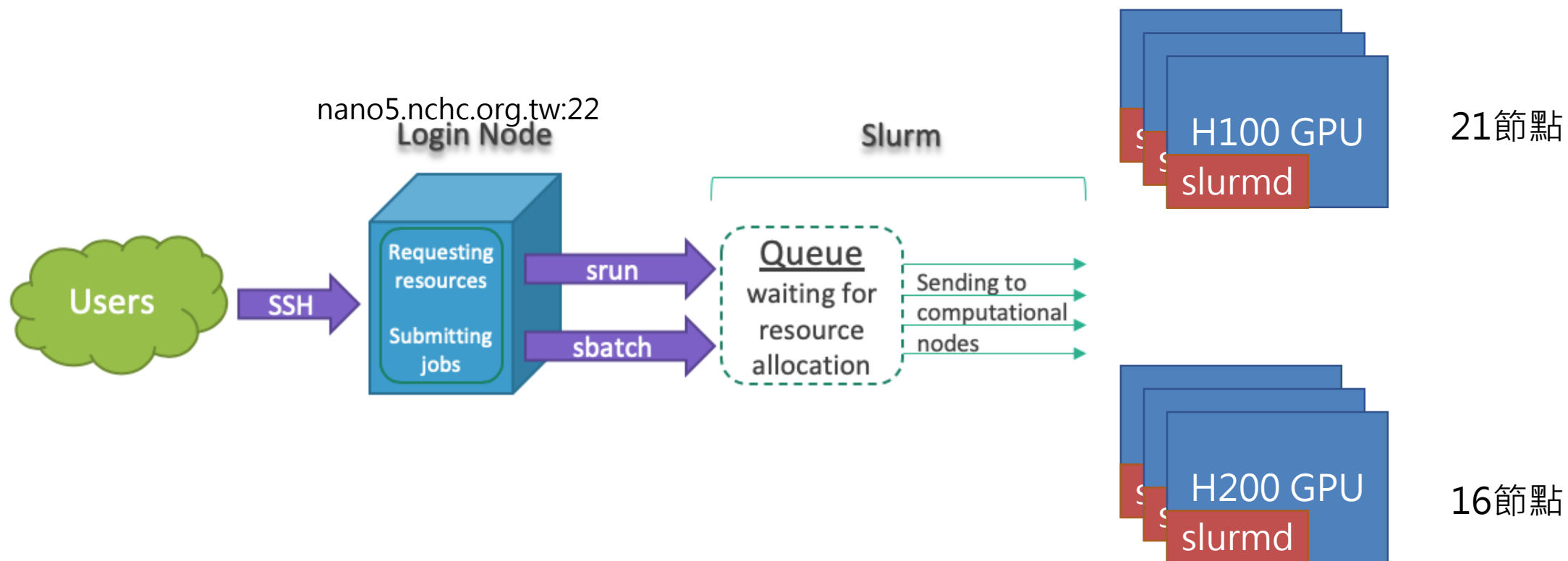
Slurm 作為資源調度軟體，為一容錯度高、高可擴展的叢集管理系統，適用於大、小型的 Linux 叢集，至今已成為許多超級電腦所使用的資源管理器。Slurm 本身並不需要針對系統內核(kernel)做調整，並且相對獨立。作為一個叢集管理系統，Slurm 提供三個關鍵功能：

- 可在一段時間內替用戶分配計算資源的獨佔或非獨佔使用
- 提供管理框架，於計算節點上進行資源的分配、執行與監控
- 可透過排程管理的工作列隊 (Queueing) 來仲裁資源分配

Script

由於 HPC 的叢集任務具備一些特殊性，要發揮高效能運算的優勢，必須要支援平行運算，或者是在叢集上運行不相互依賴的分布式作業。

Slurm 簡介



Slurm 簡介

Slurm (Simple Linux Utility for Resource Management) 在每一個計算節點上面都有運行一個Slurm daemon程式，簡稱slurmd。slurmd 用於管理計算節點，包含監控該節點上運行的任務，接受來自控制器 (Controller)的任務，以及將該任務分派到計算節點上。如果控制器發出請求，slurmd 也可以停止進行該任務。

Slurm 基礎指令

1. **sinfo**：用於查看節點資訊，是最簡單的查詢方式之一。執行 sinfo 指令會列出目前系統中所有分區（partition）及節點的基本狀態。如果只是想快速了解可用節點的整體情況，使用 sinfo 即可取得足夠資訊。
2. **sbatch**：用於提交作業腳本，供之後排程執行。通常提交的腳本會包含一個或多個 srun 指令，用來啟動並行任務，因此是 Slurm 中最基本的作業提交方式之一。sbatch 會將批次腳本提交給 Slurm 系統執行，腳本內容可透過檔案指定；若未提供檔案名稱，則會從標準輸入（standard input）讀取內容。在批次腳本中，也可以使用以 #SBATCH 開頭的註解方式，在執行前設定相關參數。
3. **squeue**：用於顯示作業或作業集合的狀態。支援多種篩選、排序與格式化選項。預設情況下，會先依優先權顯示正在執行的作業，再顯示等待中的作業，因此是最常用的作業查詢指令之一。
4. **scancel**：用於取消正在等待或執行中的作業或作業集合，也可對執行中的作業或其相關程序發送任意訊號。可用於終止特定作業、作業陣列（job array）或作業步驟（job step）。
5. **sacct**：用於查詢帳號相關的作業歷史資訊，例如正在執行、已終止或已完成的作業狀態。是用來查看作業紀錄與歷史狀態的基本指令之一。

Slurm 基礎指令

6. **srun**：用於提交任務或作為提交任務集中的單一任務。srun有多種選項來指定資源需求，包括：最小和最大節點數，CPU 數，是否使用特定節點，以及特定節點的特徵（例如 Memory 大小，硬碟空間，某些必需的功能等）。一個任務可以包含多步驟的任務，並可依照被分配的節點資源，依序或平行在獨立或共享的節點上運作。
7. **salloc**：用於即時分配資源分配，通常會用於分配資源並且產生一個 shell，然後使用 shell 執行 srun 的命令來啟動並執行任務。
8. **scontrol**：用於查看和/或修改 Slurm 狀態的管理工具。請注意，許多 scontrol 命令只能以 root 用戶身份執行。

Queue與計算資源

晶創25 提交 Job 時，可使用的 Queue (Partition) 如下：

佇列	單一計畫 (account) 可用 GPU 總數	單一工作 (job) 最長執行時間 (小時)	同一時間內每位用戶 (user)		備註 (GPU 類型)
			可執行 (running) 工作總量	可等候 (pending) 工作總量	
dev	8	2	2	-	H100
normal	16	48			H100
normal2	16	48			H200
4nodes	32	12			H100

註1：queue依實務需要會調整變動，實際queue清單請依實務操作取得

註2：系統限制每張 GPU 最多可使用 12 CPU cores、200 GB 記憶體，以確保資源公平分配。(如需使用300GB 記憶體，需搭配2 GPU數)

節點狀態查詢

sinfo -s

```
[twsutwx533@rms01 root]$ sinfo -s
PARTITION AVAIL  TIMELIMIT  NODES(A/I/O/T) NODELIST
dev        up      2:00:00    10/1/0/11  hgpn[01-06,17-21]
normal     up    2-00:00:00  10/0/0/10  hgpn[02-06,17-21]
4nodes     up    1-00:00:00  10/0/0/10  hgpn[02-06,17-21]
normal2     up    2-00:00:00    6/1/0/7  hgpn[39-41,43-46]
```

節點狀態列表-常用

IDLE	表示節點目前可用，但尚未被分配任何任務。
MIXED	表示節點部分資源已被分配使用，仍有部分資源可用。
COMPLETEING	表示該節點上的所有任務皆已進入完成流程中。Slurm 允許在任務結束後執行額外的清理或收尾程序，當所有任務與後續程序皆完成後，此狀態會被移除。
DRAINED	表示節點不再接受新的資源分配請求，通常由系統管理員手動設定，用於維護或下線準備。
DRAINING	表示節點目前仍在執行現有任務，但完成後將轉為 DRAINED 狀態，不再接受新任務。
FAIL	表示節點因發生問題而無法使用，不再接受資源分配。此狀態與 DRAINED 不同，通常代表異常或故障。
DOWN	表示該節點不可使用，例如硬體故障或系統異常。系統可能自動或由管理員手動將節點設為此狀態；當節點恢復正常後，Slurm 會將其重新加入可用資源中。
ALLOCATED	表示該節點已被一個或多個任務分配使用。
PLANNED	此節點已被規劃預留給較高優先順序的 Job 優先權。

節點狀態列表

ALLOCATED+	表示該節點已被分配給任務使用，且部分任務正處於完成 (COMPLETING) 階段。
FAILING	表示節點目前仍在執行任務，但因發生問題，將在特定時間後停止接受新的資源分配。
FUTURE	表示節點尚未完成初始化或配置，預計在未來某個時間點才會啟用。
MAINT	表示節點處於維護或保留狀態，暫時不可使用。
REBOOT	表示節點正在進行重新啟動流程。
PERFCTRS(NPC)	表示節點正在進行效能監控 (performance counters) 用途，因此暫時無法提供給一般作業使用。
POWER_DOWM	表示節點已關閉電源，無法執行任何作業，通常由節能機制控制。
POWER_UP	表示節點正在開機中，由節能或電源管理機制啟動。
RESERVED	表示節點已被高優先權或預約任務保留，暫時無法提供一般資源使用。
UNKNOWN	表示 Slurm 控制器尚在初始化或無法確認節點狀態。

節點狀態列表

狀態分類

- **空閒**：IDLE
- **工作中(工作結束後可繼續處理已排程任務)**：ALLOCATED、ALLOCATED+、COMPLETING、MIXED
- **工作中(工作結束後會轉為不可用狀態)**：DRAINING、FAILING
- **不可用**：DOWN、DRAINED、FAIL、FUTURE、MAINT、REBOOT、PERFCTRS、POWER_DOWM、POWER_UP、RESERVED，以及含「#」、「\$」、「@」任一字符
- **未知**：UNKNOWN

節點狀態列表

圖中的狀態，目前partition dev可用節點(NODE)有幾個？

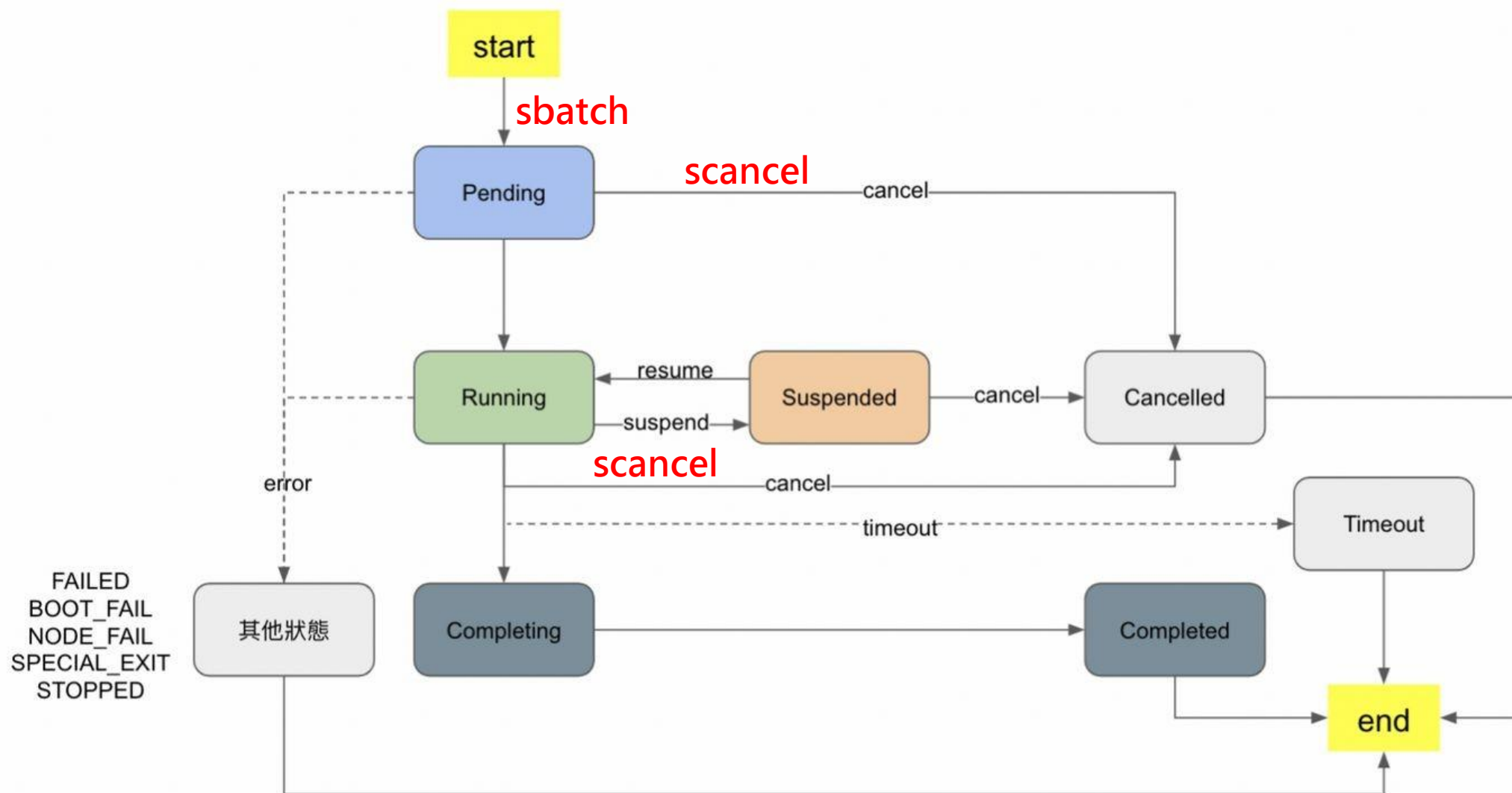
```
[twsutwx533@rms01 root]$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
dev        up      2:00:00    10    mix  hgpn[02-06,17-21]
dev        up      2:00:00     1    idle hgpn01
normal     up    2-00:00:00    10    mix  hgpn[02-06,17-21]
4nodes     up    1-00:00:00    10    mix  hgpn[02-06,17-21]
normal2    up    2-00:00:00     6    mix  hgpn[39-41,43,45-46]
normal2    up    2-00:00:00     1    idle hgpn44
```


節點狀態列表

查看完整節點狀態資訊
sinfo --long

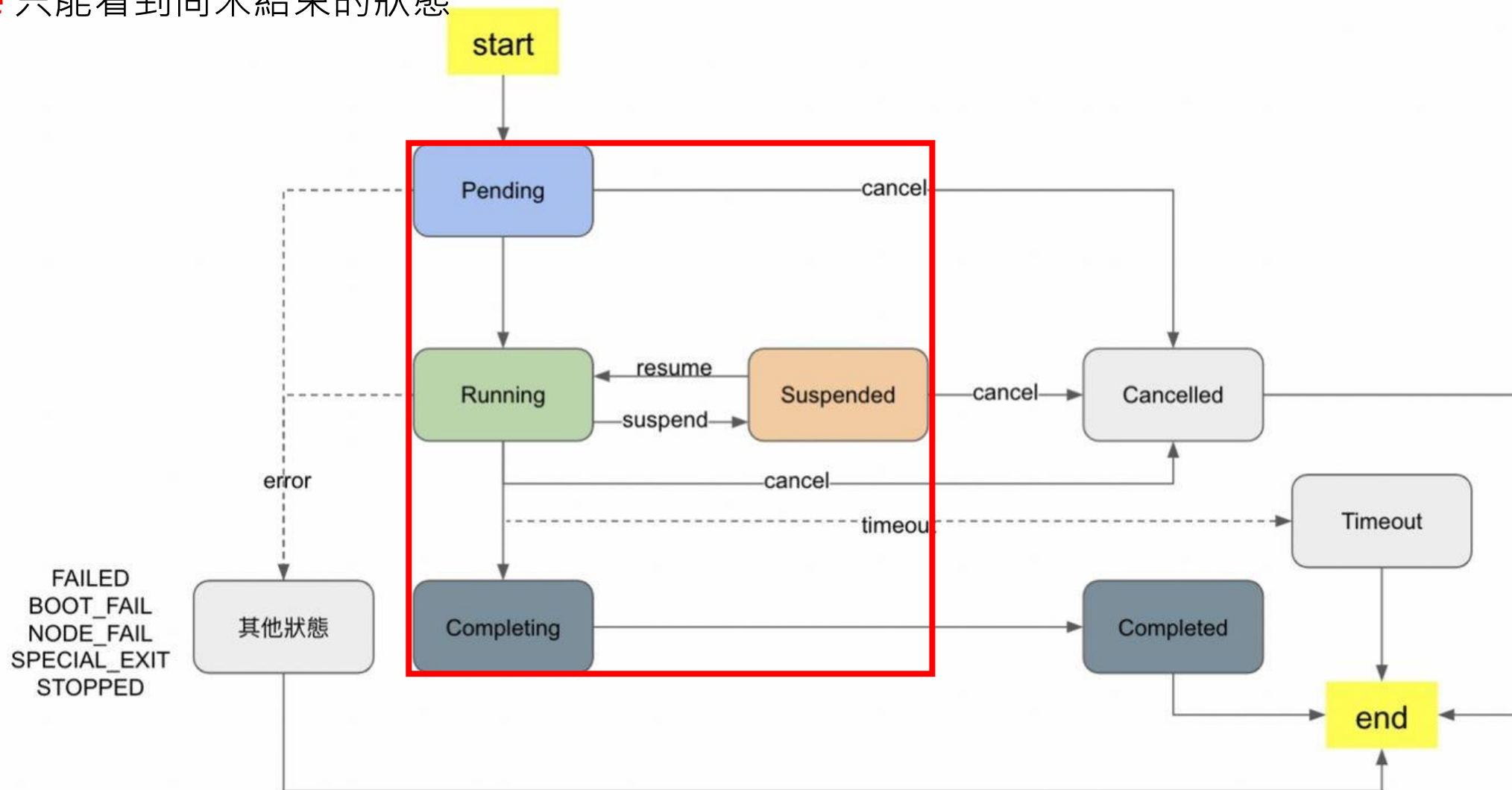
```
[twsutwx533@rms01 root]$ sinfo --long
Thu Aug 13 14:16:22 2026
PARTITION AVAIL  TIMELIMIT  JOB_SIZE  ROOT  OVERSUBS      GROUPS  NODES  STATE  RESERVATION  Nodelist
dev        up      2:00:00      1      no      NO          all     10     mixed  hgpn[02-06,17-21]
dev        up      2:00:00      1      no      NO          all      1     idle   hgpn01
normal     up    2-00:00:00    1-2    no      NO          all     10     mixed  hgpn[02-06,17-21]
4nodes     up    1-00:00:00    1-4    no      NO          all     10     mixed  hgpn[02-06,17-21]
normal2    up    2-00:00:00    1-2    no      NO          all      6     mixed  hgpn[39-41,43,45-46]
normal2    up    2-00:00:00    1-2    no      NO          all      1     idle   hgpn44
```

任務生命週期



任務生命週期

queue 只能看到尚未結束的狀態



任務狀態範例

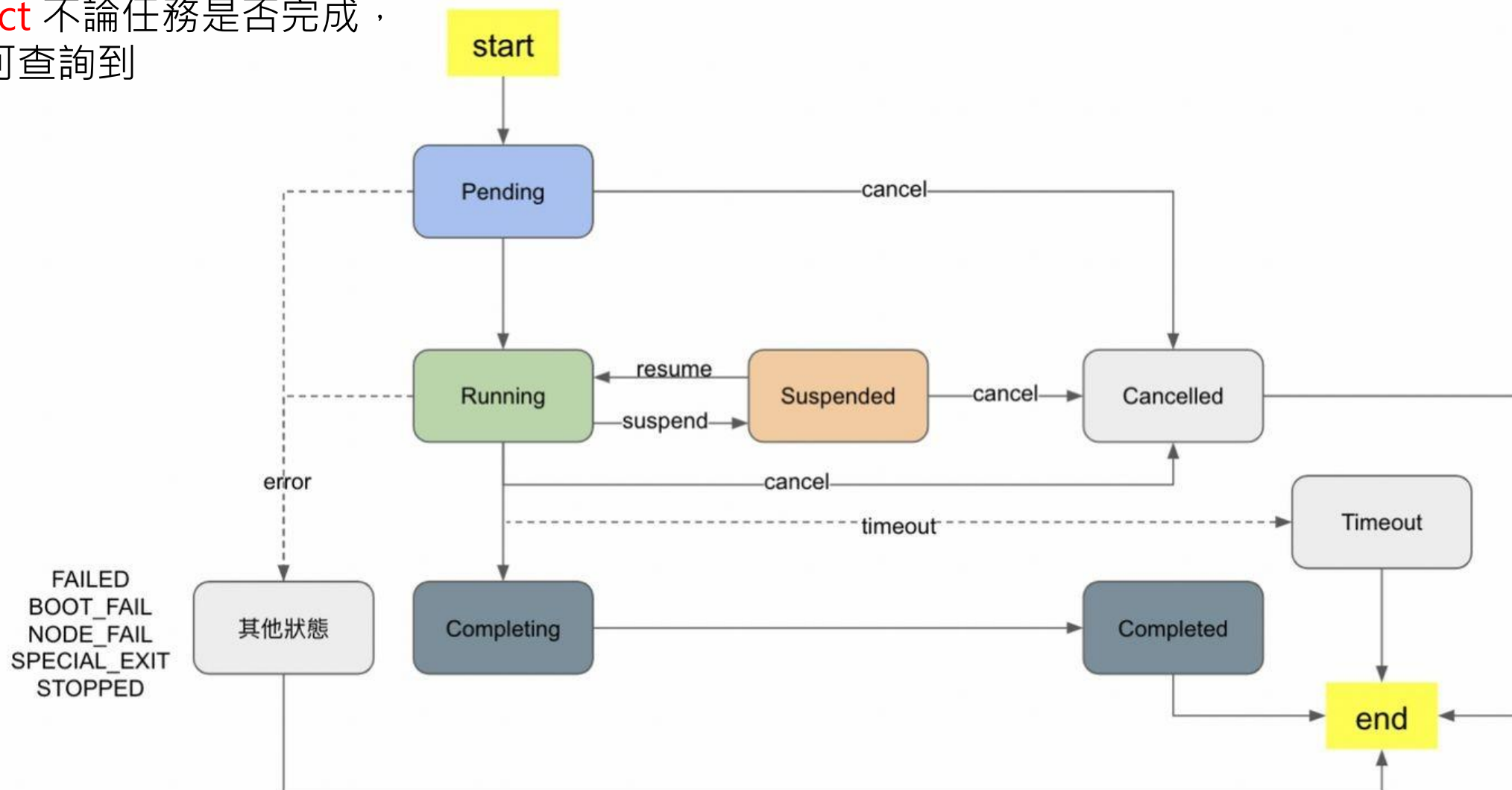
```
[xxx@cbi-lgn01 ~]$ squeue
```

JobID	Partition	JobName	User	State	Time	Nodes	NodeList / Reason
301165	dev	vllm-ser	xxx	R	31:15	1	hgpn04
300950_[2-6]	dev	OffGridS	xxx	PD	0:00	1	(Priority)
301198	dev	HETRANK	xxx	PD	0:00	1	(Priority)

編碼縮寫	狀態名	說明
PD	PENDING	任務正在等待資源被分配
R	RUNNING	任務正在運行中

任務生命週期

sacct 不論任務是否完成，皆可查詢到



任務狀態對照表-常用

編碼縮寫	狀態名	說明
PD	PENDING	任務正在等待資源被分配
R	RUNNING	任務正在運行中

任務狀態對照表

編碼縮寫	狀態名	說明
BF	BOOT_FAIL	任務因啟動失敗而中止，通常是因為硬體故障
CA	CANCELLED	任務被用戶或系統管理員終止，該任務可能已經運行或還沒運行
CD	COMPLETED	任務的每個處理程序在計算節點都已執行完成，且沒有出現問題
CF	CONFIGURING	任務已被分配資源，正在等待所有資源準備就緒，通常這個狀態僅出現一瞬間
CG	COMPLETING	任務正在完成中，若同時有多個節點在處理此程序，某些節點可能還在運行
F	FAILED	若ExitCode不為「0」，則表示有其他錯誤讓此任務中止
NF	NODE_FAIL	任務因在一個或多個節點的硬體故障而中止
PR	PREEMPTED	任務由於資源被強佔而中止
RV	REVOKED	任務已轉移到其他運算叢集上
SE	SPECIAL_EXIT	特殊狀態下重新排隊
ST	STOPPED	任務已被分配資源後運行，但執行過程中接收到 SIGSTOP 而停止，該任務將 繼續保有原本的資源配置
S	SUSPENDED	任務在執行的過程中被暫停，所使用的資源釋放給其他任務使用
TO	TIMEOUT	任務運行的時間超過設置的限制

撰寫slurm腳本常用名詞

1. **計畫代號 (Account)**：Slurm Account在Nano5代表iService計畫代號，提交工作前必須加入iService有效計畫及錢包餘額為正數。
2. **節點 (Node)**：Slurm 大致可區分為三個節點角色：
 - a. 管理節點 (Head Node)：管理Slurm叢集系統、工作與排程等。
 - b. 工作派送節點 (Submit Node)：供用戶登入提交與管理計算工作的節點，如登入節點。
 - c. 計算節點 (Worker Node)：工作派送後，實際執行程式運算的節點，如指令sinfo的NODELIST欄位就是計算節點清單。
3. **佇列 (Partition)**：將多個計算節點組成一個集合，若計算節點的硬體配置不同，透過佇列 (Partition) 可以幫助使用者選擇合適的節點並派送工作進行運算。
4. **工作 (Job)**：工作是在一個或多個計算節點上執行相同的程式。用戶可以通過提交一個工作給SLURM管理節點，然後SLURM管理節點會分配資源給該工作，並在適當的計算節點上執行。
5. **工作識別碼 (Job ID)**：在Slurm中提交的工作都會被分配一個唯一的識別碼，稱為"job ID"。這個ID是由SLURM系統自動產生的，用戶可以使用這個ID來檢查工作的狀態、終止工作或查詢工作的詳細資訊。

slurm常用名詞

6. **工作名稱 (Job Name)**：是用戶指定的一個用於識別工作的名稱。工作名稱可以是任何用戶選擇的字串，通常用於方便用戶識別和管理提交的工作。
7. **任務 (Tasks)**：每個task代表一個要在計算節點上執行的工作。在平行計算中，tasks通常用於執行同一個任務的多個實例，從而加速計算的進行。
8. **任務資源 (Task Resource)**：指Job在計算節點上執行時所需的資源配置，包括節點數量、每個任務使用的CPU/GPU核心數量和記憶體大小等。以下是幾個與任務資源相關的SLURM常用參數：
 - A. `-N, --nodes`：這個參數用於指定Job所需的計算節點數量。
 - B. `-c, --cpus-per-task`：這個參數用於指定每個任務 (task) 所使用的CPU核心數量。通常，Job會被分配到一個或多個計算節點上，而每個計算節點上可能有多個CPU核心。透過這個參數，可以控制每個任務在節點上可使用的CPU核心數。
 - C. `--ntasks-per-node=<ntasks>`：用來指定每個節點上執行的任務數量。透過這個參數，您可以控制每個節點上平行計算的任務數量。若不指定此參數，則任務數量預設值為1，最大值為計算節點的可用CPU核心數。
 - D. `--gres=gpu:<num>`：用於指定Job所需GPU數量的參數
9. **最大執行時間 (Wall Time)**：指Job的最大執行時間，也就是Job可以在計算節點上運行的時間限制。由Slurm用來限制作業運行時間，以確保系統資源的有效利用和作業的及時完成。

撰寫slurm腳本基本概念

國網中心slurm管理，建立一些基本參數需加入，驗證後才放行submit job

必要job submission 參數		
1.	計畫代號 (Account)	-A, --account = <account> iService計畫代號，提交工作前必須加入iService有效計畫及錢包餘額為正數
2.	佇列 (Partition)	-p, --partition = <partition> 將多個計算節點組成一個集合，透過佇列 (Partition) 使用者可選擇合適的節點並派送工作進行運算
3.	GPU任務資源	--gpus-per-node=<number>

撰寫slurm腳本基本概念

建議可加入job submission 參數

1.	節點 (Node)	-N, --nodes = <minnodes>
2.	工作識別碼 (Job ID)	-j, --jobs = <job_id>
3.	工作名稱 (Job Name)	-J, --job-name = <job_name>
4.	任務 (Tasks)	-n, --ntasks = <number>
5.	任務資源 (Task Resource)	-c, --cpus-per-task & -ntasks-per-node = <ntasks>
6.	最大執行時間 (Wall Time)	-t, --time= <time>
7.	寄信通知 (Mail Type)	-mail-type = <mail-type> (NONE, BEGIN, END, FAIL, ALL)
	與設定收信者 (Mail User)	-mail-user = <u>user@mail.com</u>

slurm腳本範例

國網中心slurm管理，建立一些基本參數需加入，驗證後才放行submit job

test01.sh

```
#!/bin/bash
```

```
#SBATCH --job-name=JAFF_Test
```

```
#SBATCH --nodes=1
```

```
#SBATCH --ntasks-per-node=1
```

```
#SBATCH --gres=gpu:1
```

```
#SBATCH --time=00:10:00
```

```
#SBATCH --account=YourProject
```

```
#SBATCH --partition=dev
```

```
## job name
```

```
## 索取 1 節點
```

```
## 每個節點運行 1 srun tasks
```

```
## 每個節點索取 1 GPUs
```

```
## 最長跑 10 分鐘 (測試完這邊記得改掉，或是直接刪除該行)
```

```
## PROJECT_ID 請填入計畫ID(ex: GOVXXXXXX)，扣款也會根據此計畫ID
```

```
## get from sinfo
```

```
echo "Hello World!"
```

```
date
```

```
hostname
```

```
sleep 60
```

```
echo "Bye"
```

slurm執行及檢查範例

國網中心slurm管理，建立一些基本參數需加入，驗證後才放行submit job

執行

```
sbatch test01.sh
```

```
[xxxx@cbi-lgn01 ~]$ sbatch test01.sh
```

```
Submitted batch job 323231
```

查看queue狀態

```
squeue | grep job id
```

```
[xxxx@cbi-lgn01 ~]$ squeue | grep 323238
```

```
323238      dev JAFF_Tes asusclou R    0:12    1 hgpn21
```

查看歷史執行記錄

```
sacct | grep job id
```

```
[xxxx@cbi-lgn01 ~]$ sacct | grep 323238
```

```
323238      JAFF_Test      dev gov110011      1 COMPLETED    0:0
```

查看queue狀態

```
squeue | grep job id
```

```
[xxxx@cbi-lgn01 ~]$ squeue | grep 323238
```

```
[xxxx@cbi-lgn01 ~]$
```

Lab 2：提交第一個 job

1. 查看節點狀態 (hint: sinfo)
2. 編寫要提交的script (hint: test01.sh)
3. 提交 job (hint: sbatch)
4. 取消 job (hint: scancel)
5. 查看歷史 job 狀態 (hint: sacct)
6. 查看 job 輸出 log (hint: cat slurm....out)

互動式任務

使用「sbatch」是最常見的使用方式，但是腳本中的某一行出現問題，就必須要反覆修改腳本 才能夠順利跑完，這對於測試非常不方便，又或者所執行的內容需要與standout 進行互動才 能夠跑的通

salloc - 向slurm allocate資源

srun - 在salloc的shell中提交子任務Job Step

互動式任務

```
[XXX@cbi-lgn01 ~]$ salloc -N1 -n1 -A YourProject -p dev --gpus-per-node=1  
salloc: Granted job allocation 323349  
salloc: Nodes hgpn01 are ready for job
```

```
[XXX@cbi-lgn01 salloc_305489 ~]$ hostname  
cbi-lgn01
```

```
[XXX@cbi-lgn01 salloc_305489 ~]$ srun hostname  
hgpn01
```

```
[XXX@cbi-lgn01 salloc_305489 ~]$ exit  
exit  
salloc: Relinquishing job allocation 305489  
salloc: Job allocation 305489 has been revoked.  
[XXX@cbi-lgn01 ~]$
```


Lab 3：使用互動式任務

1. 使用salloc索取一個節點資源
2. 使用srun執行程式
3. 退出任務

注意事項

1. 每個計算工作至少指定 1 張 GPU，且不能超過 1 GPU : 12 CPU : 200 GB Memory 比例
2. 當用戶在登入節點，因避免系統超過負載，運行超過 5 分鐘的 GPU process，將會自動清除該用戶所有 process。
3. 在登入節點當機時，使用 srun and salloc 指令提交 job 可能會造成計算中斷或資料流失，因此建議使用 sbatch 指令運行大規模計算工作。
4. queue 與節點數量會定期檢討與調整。
5. 管理員會定期清理登入節點的 /tmp 目錄以維持系統運作，請避免於 /tmp 存放重要資料。
6. 若需搭配較優先或時間較長的計算工作的執行時間 (walltime)，請洽詢系統管理員或客服人員。

大綱

- 課前準備
- HPC功能介紹
- Slurm 簡介
- **Module 概觀**
- Miniconda 介紹
- Apptainer / Singularity容器介紹
- 跨節點運算跟單節點運算
- Q & A 與補充

Module 概觀

環境變數管理工具

在 HPC 使用情境中，當編譯 (compile) 或是執行應用程式時，常會有複雜的工具或函式庫 (library) 之間的相依性，因此常使用 Module 作為管理環境變數 (environment variable) 的工具，方便在不同的工具間轉換環境變數。

使用 Module 後，可以輕鬆切換，無需再使用 `export` 指令切換環境變數以使用特定軟體套件、工具函式庫，且不需要再透過 `unset` 移除環境變數，只要使用 `module purge` 以一個指令即可完全清除。

Module 基本指令

說明	指令	快速記憶指令
列出可使用的 modules	<code>module avail</code>	<code>ml av</code>
列出已套用的 modules	<code>module list</code>	
查看 modules 的說明	<code>module whatis package1 package2 ...</code>	

Module 基本指令

說明	指令	快速記憶指令
套用指定的modules	<code>module load package1 package2 ...</code>	<code>ml load/add <package></code>
移除指定的 modules	<code>module unload package1 package2 ...</code>	<code>ml unload <package></code>
移除全部的 modules	<code>module purge</code>	<code>ml purge <package></code>

Lab 4 : 使用module

1. 列出平台可使用的 modules (hint: `module avail`)
2. 套用指定的 module (hint: `module list/ module load`)
3. 列出已套用的 modules (hint: `module list`)
4. 移除全部的 modules (hint: `module purge`)

大綱

- 課前準備
- HPC功能介紹
- Slurm 簡介
- Module 概觀
- **Miniconda 介紹**
- Apptainer / Singularity容器介紹
- 跨節點運算跟單節點運算
- Q & A 與補充

Miniconda 簡介

Miniconda 是一個輕量級的 Anaconda 版本，它是 Python 的套件管理器和環境管理器。與完整的 Anaconda 發行版相比，Miniconda 的安裝包更小，只包含 Conda 套件管理器和一些基本的套件。它允許用戶自定義安裝所需的套件，而不需要下載整個 Anaconda 發行版。

使用 Miniconda，可以輕鬆地創建、管理和刪除 Python 環境，並方便地安裝、更新和刪除各種 Python 套件。這對於管理多個 Python 項目並保持它們的相依性的清晰性和一致性非常有用。

Miniconda 提供了一個輕量級、靈活和自定義的方式來管理 Python 環境和套件，使開發人員能夠更有效地處理 Python 專案的依賴關係和管理。

Miniconda 的使用原因

- 無需管理員權限即可安裝
- 能解決環境相依與版本衝突問題
- 輕量且彈性高

如何使用 Miniconda ?

- 透過 `module load miniconda3` 載入 conda 環境。
- 透過 `module purge` 即可卸載 conda 環境。
- 使用前建議移除之前使用 Conda 產生的 `conda initialize` 內容 (移除指令 `conda init --reverse`)。

Conda 操作範例

1. 載入Conda並建立虛擬環境

```
# 清除 module 確保環境乾淨
[xxxx@cbi-lgn01 ~]$ module purge

# 載入 Conda 環境，以預設 python3 的 miniconda3 為範例
[xxxx@cbi-lgn01 ~]$ module load miniconda3/24.11.1
(base) [xxxx@cbi-lgn01 ~]$

# 由於 晶創25 設計的 miniconda module，會自動初始化環境變數，不需要再 conda init。
# 建議移除之前 conda init 產生的內容
(base) [xxxx@cbi-lgn01 ~]$ conda init --reverse

# 創立 Conda 虛擬環境 (範例名稱為 mytf_cpu)，並安裝 Anaconda 公司已打包好的 tensorflow-gpu。
# 該環境已經包含 CUDA 和 cuDNN，無須再自行安裝
(base) [xxxx@cbi-lgn01 ~]$ conda create --name mytf_cpu python=3.10 pytorch

# 進入此虛擬環境
(base) [xxxx@cbi-lgn01 ~]$ conda activate mytf_cpu
(mytf_cpu) [xxxx@cbi-lgn01 ~]$
```

Conda 操作範例

2. 在conda中執行程式

```
# Sample Python Code
```

```
# sample_job.py
```

```
import sys
```

```
import torch
```

```
print("--- Job 開始執行 ---")
```

```
print(f"目前使用的 Python 路徑: {sys.executable}")
```

```
print(f"PyTorch 版本: {torch.__version__}")
```

```
print(f"是否能使用 CUDA (GPU): {torch.cuda.is_available()}")
```

```
print("--- Job 順利結束 ---")
```

```
# 執行python程式
```

```
(mytf_cpu) [xxxx@cbi-lgn01 ~]$ python sample_job.py
```

Conda 操作範例

3. Conda建立後基本操作指令

查看所有環境

```
(mytf_cpu) [xxxx@cbi-lgn01 ~]$ conda env list
```

安裝套件

```
(mytf_cpu) [xxxx@cbi-lgn01 ~]$ conda list pandas
```

```
(mytf_cpu) [xxxx@cbi-lgn01 ~]$ conda install pandas=2.3.1
```

```
(mytf_cpu) [xxxx@cbi-lgn01 ~]$ conda list pandas
```

移除套件

```
(mytf_cpu) [xxxx@cbi-lgn01 ~]$ conda remove pandas
```

從指定社群安裝套件

```
(mytf_cpu) [xxxx@cbi-lgn01 ~]$ conda install -c conda-forge pandas #指定 channel
```

更新套件

```
(mytf_cpu) [xxxx@cbi-lgn01 ~]$ conda update -c conda-forge pandas
```

```
(mytf_cpu) [xxxx@cbi-lgn01 ~]$ conda update --all
```

Conda 操作範例

4. 離開 conda 並保留環境

```
# 離開 mytf_gpu 虛擬環境
(mytf_cpu) [xxxx@cbi-lgn01 ~]$ conda deactivate
(base) [xxxx@cbi-lgn01 ~]$

# 卸載 miniconda3 module 環境
(base) [xxxx@cbi-lgn01 ~]$ module unload miniconda3/24.11.1
[xxxx@cbi-lgn01 ~]$

# 載入 Conda 環境，以預設 python3 的 miniconda3 為範例
[xxxx@cbi-lgn01 ~]$ module load miniconda3/24.11.1
(base) [xxxx@cbi-lgn01 ~]$

# 進入此虛擬環境
(base) [xxxx@cbi-lgn01 ~]$ conda activate mytf_cpu
(mytf_cpu) [xxxx@cbi-lgn01 ~]$
```

Conda 操作範例

5. 刪除 Conda 殘留檔案

```
# 離開 mytf_gpu 虛擬環境
(mytf_cpu) [xxxx@cbi-lgn01 ~]$ conda deactivate
(base) [xxxx@cbi-lgn01 ~]$

# 刪除 mytf_cpu 虛擬環境檔案
(base) [xxxx@cbi-lgn01 ~]$ conda remove --name mytf_cpu --all

# 卸載 miniconda3 module 環境
(base) [xxxx@cbi-lgn01 ~]$ module unload miniconda3/24.11.1
[xxxx@cbi-lgn01 ~]$
```

Lab 5 : slurm job script with conda

1. 使用 miniconda3 module 建立conda環境 (hint: conda create --name mytf_cpu python=3.10 pytorch)
2. 啟用conda環境 (hint: conda activate mytf_cpu)

3. 提交 job with conda

Sample Python Code

```
# sample_job.py
import sys
import torch
```

```
print("--- Job 開始執行 ---")
print(f"目前使用的 Python 路徑: {sys.executable}")
print(f"PyTorch 版本: {torch.__version__}")
print(f"是否能使用 CUDA (GPU): {torch.cuda.is_available()}")
print("--- Job 順利結束 ---")
```

4. 離開 conda 環境並卸載 module

大綱

- 課前準備
- HPC功能介紹
- Slurm 簡介
- Module 概觀
- Miniconda 介紹
- Apptainer / Singularity容器介紹
- 跨節點運算跟單節點運算
- Q & A 與補充

Apptainer / Singularity 容器介紹

Singularity 容器是一種用於虛擬化應用程式和環境的解決方案，主要用於科學計算和高性能計算領域。與傳統的容器技術（如 Docker）不同，Singularity 容器更適用於在 HPC（High Performance Computing）環境中運行，因為它們允許用戶以與本地環境相似的方式運行應用程式，而無需特權使用者權限。

自 NGC 19.11 版本以後的容器，已支援 Singularity 容器，適用 HPC 使用情境，因此晶創25(命令列介面)用戶可以直接下載 NGC 容器使用，亦可用 NGC 提供的容器為基礎映像檔，疊加自己常用工具。

使用Singularity的原因

- 無 root 權限環境
- 環境一致性與可重現性
- 系統整合性高、資源開銷低

Singularity 容器介紹

NGC 容器 (NVIDIA GPU Cloud Container) 由 NVIDIA 提供的預建深度學習與 HPC 容器

PyTorch	https://catalog.ngc.nvidia.com/orgs/nvidia/containers/pytorch?version=26.03-py3
TensorFlow	https://catalog.ngc.nvidia.com/orgs/nvidia/containers/tensorflow?version=25.02-tf2-py3-igpu
CUDA	https://catalog.ngc.nvidia.com/orgs/nvidia/containers/cuda?version=13.2.0-cudnn-runtime-ubuntu24.04
HPC SDK	https://catalog.ngc.nvidia.com/orgs/nvidia/containers/nvhpc?version=26.1-devel-cuda_multi-rockylinux9
TensorRT	https://catalog.ngc.nvidia.com/orgs/nvidia/containers/tensorrt?version=26.03-py3

Singularity 容器介紹

sample_job.py

```
import sys
import torch

print("--- Job 開始執行 ---")
print(f"目前使用的 Python 路徑: {sys.executable}")
print(f"PyTorch 版本: {torch.__version__}")
print(f"是否能使用 CUDA (GPU): {torch.cuda.is_available()}")
print("--- Job 順利結束 ---")
```

Singularity 容器介紹

下載 / 生成容器

```
# 配置計算節點
srun --account=YourPrj --partition=dev --gpus-per-node=1 --cpus-per-task=12 --pty
bash
# singularity pull <檔名.sif> <容器的指令>
singularity pull pytorch.sif docker://nvcv.io/nvidia/pytorch:24.02-py3
```

```
# 直接在shell 執行容器指令
# singularity exec <檔名.sif> <在容器內執行的指令>
singularity exec pytorch.sif python sample_job.py
```

```
# 在容器內執行指令
# singularity shell <檔名.sif>
singularity shell pytorch.sif
python sample_job.py
```

Singularity 容器介紹

下載 / 生成容器

配置計算節點

```
srun --account=YourPrj --partition=dev --gpus-per-node=1 --cpus-per-task=12 --pty  
bash
```

幸 # singularity pull <檔名.sif> <容器的指令>

```
singularity pull pytorch.sif docker://nvcr.io/nvidia/pytorch:24.02-py3
```

直接在shell使用GPU執行容器指令

singularity exec --nv <檔名.sif> <在容器內執行的指令>

```
singularity exec --nv pytorch.sif python sample_job.py
```

指定容器使用GPU執行指令

singularity shell <檔名.sif>

```
singularity shell --nv pytorch.sif  
python sample_job.py
```

Lab 6: 使用 Singularity 執行節點運算

1. 安裝 Singularity pytorch.sif 容器

(hint 1: `srun --account=YourPrj --partition=dev --gpus-per-node=1 --cpus-per-task=12 --pty bash`)

(hint 2: `singularity pull pytorch.sif docker://nvcr.io/nvidia/pytorch:24.02-py3`)

2. 指定使用 Singularity pytorch.sif 容器，不使用GPU 執行 Python 程式 (hint: 使用 `sample_job.py`)

3. 指定使用 Singularity pytorch.sif 容器，使用GPU 執行 Python 程式 (hint: 使用 `sample_job.py`)

4. 離開 GPU 節點

大綱

- 課前準備
- HPC功能介紹
- Slurm 簡介
- Module 概觀
- Miniconda 介紹
- Apptainer / Singularity容器介紹
- 跨節點運算跟單節點運算
- Q & A 與補充

跨節點運算跟單節點運算

一、核心概念與硬體架構的差異

比較維度	單節點運算 (Single-Node)	跨節點運算 (Multi-Node / Distributed)
物理主機	所有的計算都在同一台主機內完成。	計算分布在 2 台或以上的獨立伺服器上。
記憶體共用	共用記憶體（ Shared Memory ）：所有 CPU 核心直接讀寫同一個主機板上的記憶體。	分散式記憶體（ Distributed Memory ）：每個節點有獨立的記憶體，節點 A 的 CPU 無法直接存取節點 B 的記憶體。
硬體限制	受限於單台機器的上限。	理論上無限。只要基礎網路跟得上，可以串聯成百上千台節點，獲得數萬個核心。

跨節點運算跟單節點運算

二、通訊機制與成本

1. 單節點：在單節點內，多核心（ Multi-threading ）或多進程（ Multi-processing ）間傳遞資料，是直接在記憶體中進行複製或共享指標，延遲低頻寬極大。
2. 跨節點：
 - 跨節點時，節點 A 算好的數據必須打包，透過物理網路線傳送給節點 B。即使 HPC 普遍採用昂貴的 InfiniBand (IB) 網路（頻寬可達 200Gbps~400Gbps），但比起同晶片內的記憶體存取，網路傳輸依舊慢數十到數百倍。
 - 通訊開銷（ Communication Overhead ）：如果程式設計不好，可能會花大把時間在「等待網路傳送數據」，導致加了節點速度反而變慢。

跨節點運算跟單節點運算

三、該選擇哪一種？

1. 選擇「單節點運算」的情況：
 - 資料量放得進單台機器的記憶體內。
 - 演算法內部有極其頻繁、密集的資料互動（例如複雜的圖論演算法、非線性優化），不適合拆分到網路執行。
 - 追求最高的單一核心轉換效率（**CP**值高）。
2. 選擇「跨節點運算」的情況：
 - 記憶體爆了：如氣象模擬、基因定序的資料量可能高達數十**TB**，必須把矩陣分散在不同機器的記憶體裡。
 - 時間太長：如單台 **GPU** 訓練一個 **LLM** 大語言模型可能要跑 100 年，透過 512 張 **GPU** 加速。
 - 任務容易切分

大綱

- 課前準備
- HPC功能介紹
- Slurm 簡介
- Module 概觀
- Miniconda 介紹
- Apptainer / Singularity容器介紹
- 跨節點運算跟單節點運算
- Q & A 與補充

費率參考

晶創25 (NANO 5) 非保留佇列(NRQ)費率

https://iservice.nchc.org.tw/nchc_service/nchc_service_qa_single.php?qa_code=799

HFS 儲存空間收費

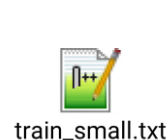
https://iservice.nchc.org.tw/nchc_service/nchc_service_qa_single.php?qa_code=802

文件參考

Nano5 使用說明

<https://man.twcc.ai/@AI-Pilot/manual#%E6%99%B6%E5%89%B5%E4%B8%BB%E6%A9%9FNano5---%E4%BD%BF%E7%94%A8%E8%AA%AA%E6%98%8E>

GPU分散式訓練腳本 # train_small.py 小型計算



```
import os
import time
import torch
import torch.nn as nn
import torch.distributed as dist
```

初始化 Distributed Process Group

```
# backend=nccl 代表 GPU 間通訊使用 NCCL
dist.init_process_group("nccl")
```

取得目前 Process 對應的 GPU 編號

```
# torchrun 啟動時會自動帶入 LOCAL_RANK
local_rank = int(os.environ["LOCAL_RANK"])
```

將目前 Process 綁定到指定 GPU

```
torch.cuda.set_device(local_rank)
```

```
model = nn.Sequential(
    nn.Linear(1024, 1024),
    nn.ReLU(),
    nn.Linear(1024, 1024)
).cuda()
```

使用 DistributedDataParallel (DDP)

```
# 在 backward 時會透過 NCCL 自動同步各 GPU 的 Gradient
model = torch.nn.parallel.DistributedDataParallel(
    model,
    device_ids=[local_rank]
)
```

```
batch_size = 64
```

建立隨機訓練資料

```
x = torch.randn(batch_size, 1024).cuda()
y = torch.randn(batch_size, 1024).cuda()
```

損失函數

```
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters())
```

等待所有 GPU 工作完成後開始計時

```
torch.cuda.synchronize()
start = time.time()
```

執行 1000 次訓練迴圈

```
for step in range(1000):
    optimizer.zero_grad()
```

```
    pred = model(x)
    loss = criterion(pred, y)
    loss.backward()
    optimizer.step()
```

```
torch.cuda.synchronize()
end = time.time()
```

```
if dist.get_rank() == 0:
    print(f"Elapsed Time = {end-start:.2f} sec")
```

```
dist.destroy_process_group()
```

GPU分散式訓練腳本 # train_large.py 大型運算



```
import os
import time
import torch
import torch.nn as nn
import torch.distributed as dist
```

```
# 初始化 Distributed Process Group
# backend=nccl 代表 GPU 間通訊使用 NCCL
dist.init_process_group("nccl")
```

```
# 取得目前 Process 對應的 GPU 編號
# torchrun 啟動時會自動帶入 LOCAL_RANK
local_rank = int(os.environ["LOCAL_RANK"])
```

```
# 將目前 Process 綁定到指定 GPU
torch.cuda.set_device(local_rank)
```

```
model = nn.Sequential(
    nn.Linear(8192, 8192),
    nn.ReLU(),
    nn.Linear(8192, 8192),
    nn.ReLU(),
    nn.Linear(8192, 8192)
).cuda()
```

```
# 使用 DistributedDataParallel (DDP)
# 在 backward 時會透過 NCCL 自動同步各 GPU 的 Gradient
model = torch.nn.parallel.DistributedDataParallel(
    model,
    device_ids=[local_rank]
)
```

```
batch_size = 1024
```

```
# 建立隨機訓練資料
x = torch.randn(batch_size, 8192).cuda()
y = torch.randn(batch_size, 8192).cuda()
```

```
# 損失函數
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters())
```

```
# 等待所有 GPU 工作完成後開始計時
torch.cuda.synchronize()
start = time.time()
```

```
# 執行 500 次訓練迴圈
for step in range(500):
    optimizer.zero_grad()
    pred = model(x)
    loss = criterion(pred, y)
    loss.backward()
    optimizer.step()
```

```
torch.cuda.synchronize()
end = time.time()
```

```
if dist.get_rank() == 0:
    print(f"Elapsed Time = {end-start:.2f} sec")
```

```
dist.destroy_process_group()
```


單節點 sbatch GPU運算

```
#!/bin/bash
#SBATCH --account=GOV115010      # (-A) iService Project ID
#SBATCH --job-name=small_1node2gpu # (-J) Job name
#SBATCH --partition=dev          # (-p) Slurm partition
#SBATCH --nodes=1                # (-N) Maximum number of nodes to be allocated
#SBATCH --gpus-per-node=2
#SBATCH --ntasks-per-node=1      # Maximum number of tasks on each node
#SBATCH --time=01:00:00          # (-t) Wall time limit (days-hrs:min:sec)
#SBATCH --output=job-%j.out      # (-o) Path to the standard output file
#SBATCH --error=job-%j.err       # (-e) Path to the standard error file
#SBATCH --mail-type=ALL          # Mail events (NONE, BEGIN, END, FAIL, ALL)
#SBATCH --mail-user=jason.huang@twcloud.ai # Where to send mail. Set this to your email address
```

```
module load singularity
```

```
srun hostname
```

```
START_TIME=$(date +%s)
```

```
# 單節點運算
```

```
srun singularity exec --nv pytorch.sif torchrun --standalone --nnodes=1 --nproc_per_node=2 train_small.py
```

```
END_TIME=$(date +%s)
```

```
echo "Runtime: $((END_TIME-START_TIME)) sec"
```

跨節點 sbatch GPU運算

```
#!/bin/bash
#SBATCH --account=GOV115010      # (-A) iService Project ID
#SBATCH --job-name=small_2node1gpu # (-J) Job name
#SBATCH --partition=dev          # (-p) Slurm partition
#SBATCH --nodes=2               # (-N) Maximum number of nodes to be allocated
#SBATCH --gpus-per-node=1
#SBATCH --ntasks-per-node=1      # Maximum number of tasks on each node
#SBATCH --time=01:00:00          # (-t) Wall time limit (days-hrs:min:sec)
#SBATCH --output=job-%j.out      # (-o) Path to the standard output file
#SBATCH --error=job-%j.err       # (-e) Path to the standard error file
#SBATCH --mail-type=ALL          # Mail events (NONE, BEGIN, END, FAIL, ALL)
#SBATCH --mail-user=jason.huang@twcloud.ai # Where to send mail. Set this to your email address
```

```
module load singularity
```

```
srun hostname
```

```
START_TIME=$(date +%s)
```

```
# 跨節點運算
```

```
MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n1)
```

```
MASTER_PORT=29500
```

```
echo "MASTER_ADDR=$MASTER_ADDR"
```

```
srun singularity exec --nv pytorch.sif torchrun --nnodes=$SLURM_NNODES --nproc_per_node=1 --rdzv_backend=c10d --rdzv_endpoint=${MASTER_ADDR}:${MASTER_PORT} train_small.py
```

```
END_TIME=$(date +%s)
```

```
echo "Runtime: $((END_TIME-START_TIME)) sec"
```

小量記憶體耗 用運行時間

```
[twsutwx533@25a-lgn02 gpu_scaling_benchmark]$ cat job-106496.out
```

```
Currently Loaded Modules:
```

```
1) singularity/4.3.7
```

```
25a-hgpn027
```

```
Mon Jun 15 05:30:31 PM CST 2026
```

```
/work/twsutwx533
```

```
Elapsed Time = 3.13 sec
```

```
Runtime: 13 sec
```

```
[twsutwx533@25a-lgn02 gpu_scaling_benchmark]$ cat job-106540.out
```

```
Currently Loaded Modules:
```

```
1) singularity/4.3.7
```

```
25a-hgpn064
```

```
25a-hgpn063
```

```
Mon Jun 15 05:47:37 PM CST 2026
```

```
Mon Jun 15 05:47:37 PM CST 2026
```

```
/work/twsutwx533
```

```
/work/twsutwx533
```

```
MASTER_ADDR=25a-hgpn063
```

```
Elapsed Time = 18.41 sec
```

```
Runtime: 88 sec
```

```
[twsutwx533@25a-lan02 gpu_scaling_benchmark]$ sacct -i 106496 -n --format=JobID,JobName,Start,End,Elapsed,State
```

106496	small_1gpu	2026-06-15T17:30:31	2026-06-15T17:30:44	00:00:13	COMPLETED
106496.batch	batch	2026-06-15T17:30:31	2026-06-15T17:30:44	00:00:13	COMPLETED
106496.exte+	extern	2026-06-15T17:30:31	2026-06-15T17:30:44	00:00:13	COMPLETED
106496.0	hostname	2026-06-15T17:30:31	2026-06-15T17:30:31	00:00:00	COMPLETED
106496.1	date	2026-06-15T17:30:31	2026-06-15T17:30:31	00:00:00	COMPLETED
106496.2	pwd	2026-06-15T17:30:31	2026-06-15T17:30:31	00:00:00	COMPLETED
106496.3	singulari+	2026-06-15T17:30:31	2026-06-15T17:30:44	00:00:13	COMPLETED

單節點2顆GPU

```
[twsutwx533@25a-lan02 gpu_scaling_benchmark]$ sacct -i 106540 -n --format=JobID,JobName,Start,End,Elapsed,State
```

106540	small_2gpu	2026-06-15T17:47:35	2026-06-15T17:49:06	00:01:31	COMPLETED
106540.batch	batch	2026-06-15T17:47:35	2026-06-15T17:49:06	00:01:31	COMPLETED
106540.exte+	extern	2026-06-15T17:47:35	2026-06-15T17:49:06	00:01:31	COMPLETED
106540.0	hostname	2026-06-15T17:47:36	2026-06-15T17:47:37	00:00:01	COMPLETED
106540.1	date	2026-06-15T17:47:37	2026-06-15T17:47:37	00:00:00	COMPLETED
106540.2	pwd	2026-06-15T17:47:37	2026-06-15T17:47:38	00:00:01	COMPLETED
106540.3	singulari+	2026-06-15T17:47:38	2026-06-15T17:49:06	00:01:28	COMPLETED

2節點各1顆GPU

大量記憶體耗 用運行時間

```
[twsutwx533@25a-lgn02 twsutwx533]$ cat job-106570.out
```

```
Currently Loaded Modules:
 1) singularity/4.3.7
```

```
25a-hgpn018
Wed Jun 17 06:14:15 PM CST 2026
/work/twsutwx533
Elapsed Time = 10.64 sec
Runtime: 78 sec
[twsutwx533@25a-lgn02 twsutwx533]$ cat job-106571.out
```

```
Currently Loaded Modules:
 1) singularity/4.3.7
```

```
25a-hgpn170
25a-hgpn018
Wed Jun 17 06:13:46 PM CST 2026
Wed Jun 17 06:13:46 PM CST 2026
/work/twsutwx533
/work/twsutwx533
MASTER_ADDR=25a-hgpn018
Elapsed Time = 14.92 sec
Runtime: 29 sec
```

```
[twsutwx533@25a-lgn02 twsutwx533]$ sacct -j 106570 -n --format=JobID,JobName,Start,End,Elapsed,State
```

106570	large 8gpu	2026-06-17T18:14:15	2026-06-17T18:15:33	00:01:18	COMPLETED
106570.batch	batch	2026-06-17T18:14:15	2026-06-17T18:15:33	00:01:18	COMPLETED
106570.exte+	extern	2026-06-17T18:14:15	2026-06-17T18:15:34	00:01:19	COMPLETED
106570.0	hostname	2026-06-17T18:14:15	2026-06-17T18:14:15	00:00:00	COMPLETED
106570.1	date	2026-06-17T18:14:15	2026-06-17T18:14:15	00:00:00	COMPLETED
106570.2	pwd	2026-06-17T18:14:15	2026-06-17T18:14:15	00:00:00	COMPLETED
106570.3	singulari+	2026-06-17T18:14:15	2026-06-17T18:15:33	00:01:18	COMPLETED

單節點8顆GPU

```
[twsutwx533@25a-lgn02 twsutwx533]$ sacct -j 106571 -n --format=JobID,JobName,Start,End,Elapsed,State
```

106571	large 2no+	2026-06-17T18:13:45	2026-06-17T18:14:15	00:00:30	COMPLETED
106571.batch	batch	2026-06-17T18:13:45	2026-06-17T18:14:15	00:00:30	COMPLETED
106571.exte+	extern	2026-06-17T18:13:45	2026-06-17T18:14:15	00:00:30	COMPLETED
106571.0	hostname	2026-06-17T18:13:45	2026-06-17T18:13:45	00:00:00	COMPLETED
106571.1	date	2026-06-17T18:13:45	2026-06-17T18:13:46	00:00:01	COMPLETED
106571.2	pwd	2026-06-17T18:13:46	2026-06-17T18:13:46	00:00:00	COMPLETED
106571.3	singulari+	2026-06-17T18:13:46	2026-06-17T18:14:15	00:00:29	COMPLETED

2節點各4顆GPU



Thank You